TECHNICAL SPECIFICATIONS & API REFERENCE

CLABTOPro Backend

Deterministic Online Judge Architecture & Execution Engine

Comprehensive technical documentation for the Node.js/Express backend services powering the CLABTOPro coding laboratory. Covers isolated Piston Docker execution, local GCC fallback, submission concurrency queueing, multi-model AI failure analysis, and complete REST API endpoints.

BACKEND STACK HIGHLIGHTS

- Runtime: Node.js 20 LTS (ES Modules)
- Web Framework: Express 4.x
- Database: MongoDB Atlas (Mongoose ODM)
- Judge: Piston Docker + Host GCC 13
- Queue: Fair Concurrency Worker Pool

PRODUCTION SPECIFICATIONS

- Target Capacity: ~120 Students
- Concurrency Cap: 4 parallel workers
- Execution Timeout: 5000ms max
- Container: Debian Bookworm Slim + GCC
- Live Status: Verified on Render

DEPLOYMENT & METADATA

Production Base URL:	**https://clabtopro-backend-ehns.onrender.com**
Health Check Endpoint:	GET /api/execution/health
Database Engine:	MongoDB Atlas Cluster0 (TLS 1.3)
Project Developer:	Raju (College Major Project Team)

SECTION 1

Server Architecture & Initialization

1.1 Architecture & Flow

CLABTOPro backend operates as a decoupled RESTful API server. Submissions enter through the submission controller, are queued to maintain server stability, and are routed to either the isolated Piston Docker container or the local GCC sandbox. The official verdict is determined deterministically by comparing normalized stdout against expected outputs.

```
Client Request (POST /api/submissions)
    %
    % % % %º server.js (Express Application & Middlewares)
    %        % % %  cors() & express.json()
    %        % % %  authMiddleware (JWT Validation)
    %        % % %  Router Dispatch (/api/submissions)
    %
    % % % %º submissionController.js (Validation & Rate Limiting)
    %        % % %  Enqueue into submissionQueue.js (Concurrency: 4)
    %
    % % % %º judgeService.js (Deterministic Orchestration)
    %        % % %  Primary: pistonService.js (HTTP /api/v2/execute)
    %        % % %  Fallback: localGccService.js (/tmp/clabto-exec-*)
    %
    % % % %º Output Normalization & Verdict Assignment
    %        % % %  ACCEPTED | WRONG_ANSWER | COMPILE_ERROR | RUNTIME_ERROR | TIME_LIMIT
    %        % % %  Save to MongoDB (with tabSwitches & test results)
    %
    % % % %º aiEvaluationService.js (Optional Failure Guidance)
            % % %  Multi-Model Chain: gemini-flash-lite-latest -> Fallback Models
```

1.2 Application Entry: server.js

The entry point 'server.js' configures environment variables via 'dotenv', initializes MongoDB connection via 'connectDB()', sets up security middlewares, and mounts seven dedicated router endpoints:

- **/api/auth:** Authentication & profile operations (routes/authRoutes.js)
- **/api/topics:** Curriculum categories and problem classifications (routes/topicRoutes.js)
- **/api/problems:** Problem catalog, test cases, and Daily Problem (routes/problemRoutes.js)
- **/api/submissions:** Code judging, queuing, and student history (routes/submissionRoutes.js)
- **/api/progress:** Student streaks, accuracy, and leaderboard (routes/progressRoutes.js)
- **/api/admin:** Instructor oversight, student metrics, and problem management (routes/adminRoutes.js)
- **/api/execution:** Engine health check and queue metrics (routes/executionRoutes.js)

SECTION 2

Deterministic Online Judge Pipeline

2.1 The Core Judge: judgeService.js

The judge service is the central evaluation authority. It rejects simulated guessing and strictly enforces physical compilation and test case execution. It handles:

- **Pre-Flight Engine Resolution:** Calls 'checkJudgeEngineHealth()' to dynamically select Piston Docker or Local GCC.
- **Output Normalization:** Applies 'normalizeOutput(str)' to strip CRLF, lone CR, and trailing blank lines while preserving numerical content.
- **Early Compilation Halt:** If compilation fails on test #1, remaining test cases are marked 'skipped' to conserve CPU resources.
- **Deterministic Verdict Assignment:** Assigns official status: accepted, wrong_answer, compile_error, runtime_error, or time_limit.

2.2 Container Sandbox: pistonService.js

Communicates with the Piston execution engine (running inside Docker on port 2000). Code is packaged as 'solution.c' with execution limits. It captures compile output, runtime stdout, stderr, exit codes, and process termination signals.

2.3 Host GCC Runner: localGccService.js

When Piston Docker is not running (e.g. cloud host without Docker-in-Docker), this service compiles and runs C code using the host's installed GCC 13 compiler. Features:

- **Temporary Workspace Isolation:** Creates randomized temporary directories (/tmp/clabto-exec-XXXXXX) with 0o700 permissions.
- **Stream EPIPE Protection:** Attaches error listeners to child.stdin to prevent process crashes if student code exits before consuming input.
- **Watchdog Timer:** Kills runaway processes with SIGKILL after runTimeout (default 3000ms).
- **Buffer Cap:** Limits stdout/stderr collection to 64KB to avoid memory exhaustion from infinite printf loops.

SECTION 3

Concurrency Control & AI Debugging

3.1 Queue Worker Pool: submissionQueue.js

Online judges cannot handle unlimited parallel C compilations without crashing the CPU. The submission queue enforces controlled concurrency:

- **Configurable Capacity:** EXECUTION_CONCURRENCY=4 limits active GCC workers. Extra requests are queued in FIFO order.
- **Queue Overload Protection:** Rejects submissions with HTTP 503 if waiting queue exceeds 200 tasks to prevent memory leaks.
- **Non-Failing Safe Handshake:** Failed execution services reject submissions with HTTP 503 rather than falsely recording WRONG_ANSWER.

3.2 Educational AI Assistant: aiEvaluationService.js

Google Gemini is integrated strictly as an educational learning assistant, decoupled from the official judge. It is fed actual execution evidence (compiler diagnostic text, exit code, or output mismatch).

- **Prompt Constraint:** Strict system instructions prohibit the AI from changing or predicting the official verdict.
- **Hidden Test Case Redaction:** If a hidden test case fails, secret inputs and outputs are replaced with '[HIDDEN TEST CASE]' to protect exam integrity.
- **Multi-Model Failover Chain:** Prevents HTTP 503 high-demand crashes by trying: gemini-flash-lite-latest -> 3.5 -> 3.1 -> 3.6.
- **Deterministic Rule-Based Fallback:** If all cloud AI APIs fail, provides structured diagnostic hints derived directly from GCC error strings.

SECTION 4

Data Models & Mongoose Schemas

4.1 Core Schema Specifications

- **User Schema (models/User.js):** username, email, password (bcrypt), role ('student'/'admin'), currentStreak, longestStreak, lastSolvedDate.
- **Problem Schema (models/Problem.js):** title, description, topic (enum), difficulty ('Easy'/'Medium'/'Hard'), inputFormat, outputFormat, constraints, sampleInput, sampleOutput, testCases: [{ input, expectedOutput, isHidden }], publishDate, status ('draft'/'published').
- **Submission Schema (models/Submission.js):** user (Ref), problem (Ref), code, language ('c'), status (enum), testCasesPassed, totalTestCases, tabSwitches, executionTime, compilerOutput, testCaseResults, aiEvaluation, submittedAt.
- **Progress Schema (models/Progress.js):** user (Ref), totalSolved, totalAccepted, totalAttempted, currentStreak, longestStreak, lastSolvedDate.

4.2 Security: sanitizeSubmissionForStudent

Exported from 'Submission.js', this function scrubs secret inputs and expected outputs from hidden test cases before returning submission payloads to students over HTTP, preventing students from inspecting network responses to cheat.

SECTION 5

API Endpoint Specifications

5.1 REST Endpoint Mapping

- **POST /api/auth/register:** Registers a student. Body: { username, email, password }
- **POST /api/auth/login:** Authenticates credentials. Returns: { token, user }
- **GET /api/problems:** Lists published problems with solved status indicator.
- **GET /api/problems/:id:** Returns problem details and student's last submitted code.
- **POST /api/submissions:** Enqueues C code for judging. Body: { problemId, code, language: 'c', tabSwitches }
- **GET /api/submissions/my:** Retrieves authenticated student's submission history.
- **GET /api/execution/health:** Live judge engine connectivity, active compiler version, and queue statistics.
- **GET /api/admin/dashboard:** Platform summary: total students, problems, submissions, pass rates.
- **GET /api/admin/students:** Detailed student roster showing problems solved and recorded tab switch violations.

5.2 Health Check Payload Example

```
// GET /api/execution/health -> HTTP 200 OK
{
  "status": "healthy",
  "activeEngine": "local_gcc",
  "engine": {
    "connected": true,
    "cInstalled": true,
    "compiler": "gcc",
    "version": "gcc (Debian 12.2.0-14+deb12u1) 12.2.0"
  },
  "queue": {
    "activeWorkers": 0,
    "waitingTasks": 0,
    "concurrency": 4,
    "capacityAvailable": 200
  }
}
```

SECTION 6

Configuration & Deployment

6.1 Environment Variables (.env)

- **PORT:** Port number (default 3400).
- **MONGODB_URI:** MongoDB Atlas connection string with replica set options.
- **JWT_SECRET:** Private key for signing student and admin session tokens.
- **GEMINI_API_KEY:** Google Gemini API key for evidence-based failure hints.
- **GEMINI_MODEL:** Set to 'gemini-flash-lite-latest' for high throughput.
- **PISTON_URL:** Piston Docker sandbox URL (default 'http://localhost:2000').
- **ENABLE_LOCAL_GCC_FALLBACK:** Set 'true' to allow host GCC execution when Docker is offline.
- **EXECUTION_CONCURRENCY:** Maximum simultaneous C execution workers (default 4).
- **EXECUTION_TIMEOUT_MS:** Execution timeout per test case in milliseconds (default 5000).

6.2 Cloud Deployment on Render

The backend is containerized via 'backend/Dockerfile' using 'node:20-bookworm-slim' with 'gcc' and 'build-essential' pre-installed. It deploys on Render as a Web Service with zero setup. Live endpoint: https://clabtopro-backend-ehns.onrender.com